SPECIFICATIONS PLATFORM

Phase 4

Data Model

Deliverable 9 of 12

Prepared for **Moses Mkhumbuza**
Project **Malls Specs — reusable specification portal**
Contents **13 entities · 4 diagrams · 23 automated checks**
Ships as **A type-checked TypeScript package under data-model/**
Date **7 August 2026**
Status **Awaiting approval to proceed to Phase 5**

CONFIRMED FOR THIS PHASE

Reference code — free text, optional, no enforced format. An existing external numbering scheme can be entered as-is.
Category — exclusive. A specification belongs to exactly one, with tags carrying every other classification.
Revision history — field-level diffs, written automatically on every save.
Every model ships empty. verify.ts asserts it on every run.

Contents

1. **Modelling principles**
2. **The model at a glance**
   - M1 Entity relationship map
   - M2 Specification lifecycle
   - M3 The empty record
   - M4 Revision history and diffs
3. **Entities**
   - 3.1 Specification
   - 3.2 Category and SubGroup
   - 3.3 Tag
   - 3.4 SpecBlock and BlockLine
   - 3.5 MetadataField
   - 3.6 Asset
   - 3.7 Revision and FieldChange
   - 3.8 User and Role
   - 3.9 Collections
   - 3.10 Search and filter
4. **Invariants**
5. **The empty catalogue**
6. **Extensibility**
7. **Delivered files**
8. **Phase 5 scope & approval**

1. Modelling principles

Five decisions shape the whole model. Each one is a direct response to something in the reference system.

1.1 Normalised, not nested

Entities are stored by id and referenced by id. The reference system holds its entire dataset as one nested JavaScript object literal, which means a specification cannot be edited without rewriting its category, and the whole catalogue must be parsed to render one record. Here, a specification carries a categoryId, not a category.

1.2 Identity is typed

CategoryId and SpecificationId are both strings at runtime, but assigning one where the other is expected is a compile error. This costs nothing at runtime and removes an entire class of bug that reference-by-string invites.

Identifiers are minted in one function. The reference system uses Date.now(), which collides under concurrent authoring and leaks creation time into every URL.

1.3 Structure is explicit, conventions are preserved

The reference system stores specification content as arrays of strings, parsed one item per line, with a leading double asterisk marking a sub-heading. That convention works and your authors know it — so it is kept, but modelled explicitly as BlockLine records rather than left implicit in a parser. The structure now survives round-tripping.

1.4 Rules live in the model

A lifecycle drawn in a design document is decoration. STATUS_TRANSITIONS is a map, canTransition() is a function, and validateTransition() rejects anything not permitted before it can be persisted. Referential integrity works the same way — a sub-group that belongs to a different category than its specification is a validation error, not a rendering bug discovered later.

1.5 Nothing is populated

Every factory returns empty strings, empty arrays, nulls and zeros. A newly created specification carries exactly two non-empty values: its identifier, and the draft status the lifecycle has to start from. This is asserted by an automated check rather than promised in prose, because a single careless default would quietly break it.

WHAT SHIPS ALONGSIDE THIS DOCUMENT

data-model/ is a real TypeScript package. It compiles under strict mode with exactOptionalPropertyTypes and noUncheckedIndexedAccess, and it has no runtime dependencies.

npm run check runs the type-checker and 23 assertions covering emptiness, referential integrity and the lifecycle. All pass.

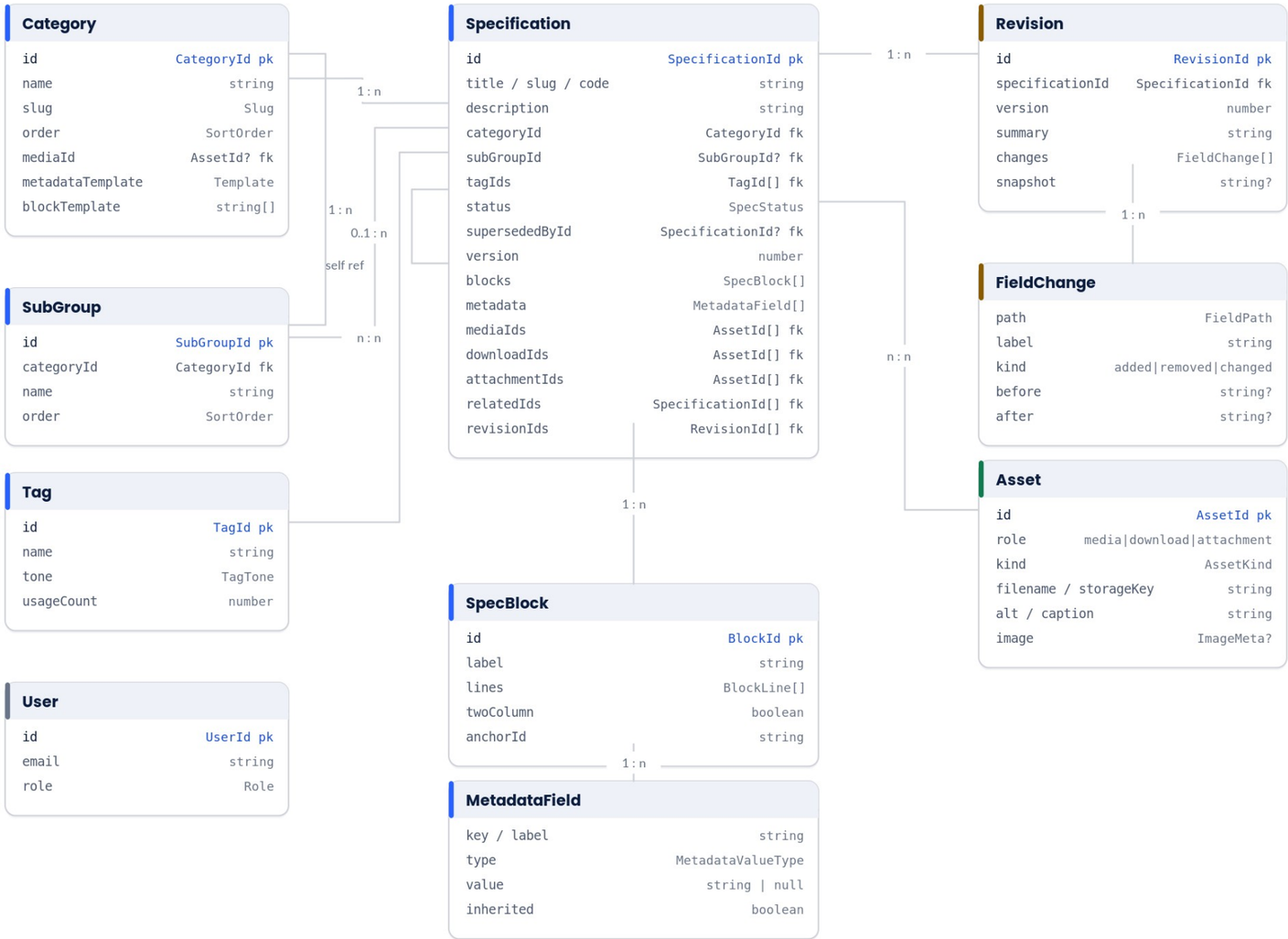
Phase 5 imports it directly. There is no translation step between this document and the build.

2. The model at a glance

Four views: how the entities relate, how a specification moves through its lifecycle, what an empty record actually contains, and how history is recorded.

M1 Entity relationship map

Entities are stored by id and referenced by id — never nested. A specification can be edited without rewriting its category, and the search index can be built without walking a tree.



1 Category to Specification is one-to-many and exclusive — a specification belongs to exactly one category, as confirmed.

2 SubGroup is the nullable middle tier. A category may expose specifications directly, which is how the reference system behaves and what the navigation component already handles.

3 Tag is many-to-many and carries no hierarchy. It is the only axis that crosses categories.

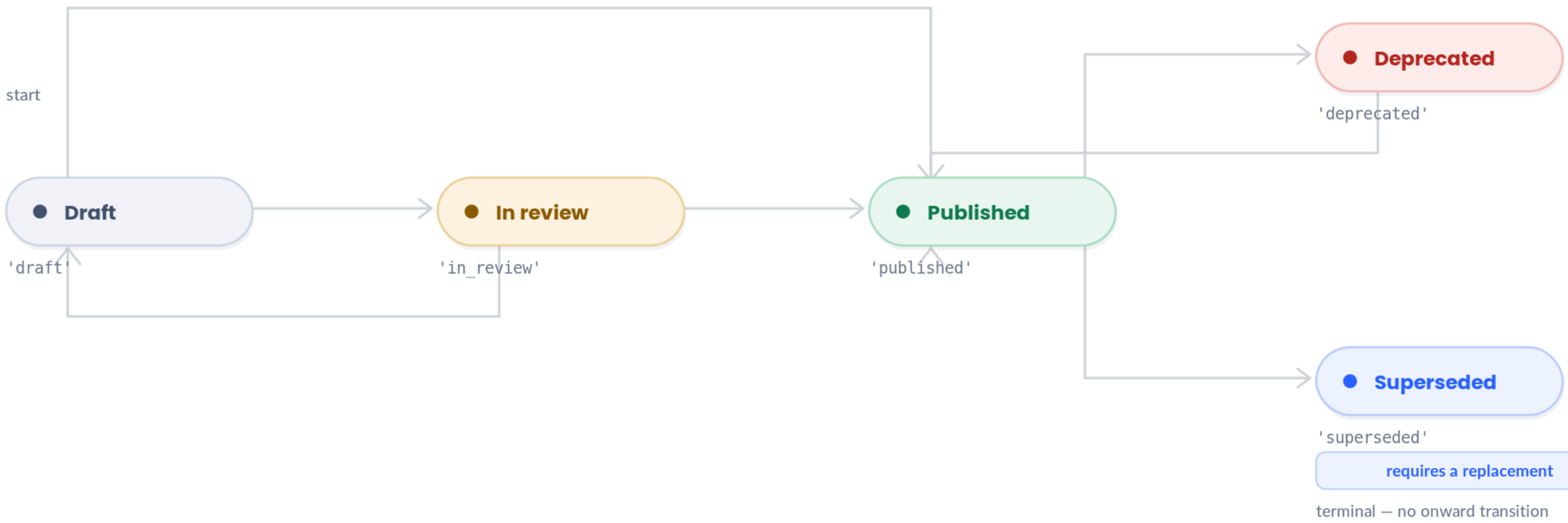
4 supersededById is a self-reference, required when status is superseded and rejected when it points at the record itself.

5 SpecBlock and MetadataField are owned by the specification, not shared. Everything else is referenced by id.

6 Asset carries a role rather than a type hierarchy, so the same file model serves the media pane, the downloads bar and the attachments section.

M2 Specification lifecycle

Seven transitions permitted, thirteen blocked. Enforced by validateTransition() before anything is persisted — the lifecycle is a rule in the model, not a diagram in a document.



WHY NOTHING RETURNS TO DRAFT

- Once published, a specification is a document other people rely on — a supplier may already be printing from it.
- Correcting it means a new revision against the published record, not a silent regression to an unpublished state.
- Deprecated can return to Published because reinstating a retired specification is a legitimate editorial act.
- Superseded cannot, because the replacement relationship would become ambiguous.

1 Five states, the confirmed vocabulary. Every new specification starts in Draft.

2 Draft can publish directly, skipping review, so a single author is not forced through an approval step that has no second party.

3 Only Published can move to Deprecated or Superseded, which prevents a draft from being retired before it ever existed publicly.

4 Superseded requires supersededById and is the only terminal state. Deprecated is reversible.

5 publiclyVisible is a property of the state, so an unauthenticated reader never sees a Draft or In-review record without a separate permission check.

M3 The empty record

What a specification looks like the moment it is created. Nothing is invented — the verification script asserts this.

createSpecification()

```
{
  id: "k3m2p8xq4a7f",
  title: "",
  slug: "",
  code: "",
  description: "",
  categoryId: null,
  subGroupId: null,
  tagIds: [],
  status: "draft",
  supersededById: null,
  version: 0,
  publishedAt: null,
  blocks: [],
  metadata: [],
  mediaIds: [],
  downloadIds: [],
  attachmentIds: [],
  relatedIds: [],
  revisionIds: [],
  currentRevisionId: null,
  searchText: "",
  createdAt: null, createdBy: null,
  updatedAt: null, updatedBy: null,
  archivedAt: null, archivedBy: null,
}
```

THE ONLY TWO NON-EMPTY VALUES

id
A minted identifier. Sortable, collision-resistant.

status: "draft"
The lifecycle has to start somewhere.

Everything else is an empty string, an empty array, a null, or a zero. No placeholder text, no lorem ipsum, no defaults borrowed from the reference system.

npm run verify — 23 checks, all passing

THE EMPTY CATALOGUE

```
{
  specifications: {},
  categories: {},
  subGroups: {},
  tags: {},
  assets: {},
  revisions: {},
  users: {},
  index: { entries: [], builtAt: null },
  schemaVersion: 1,
  generatedAt: null,
}
```

data/catalogue.empty.ts — the file the application loads on first run.

1 A newly created specification carries exactly two non-empty values: its identifier, and the draft status the lifecycle has to start from.
2 Nulls and empty strings are used deliberately rather than interchangeably. Null means 'no value exists'; an empty string means 'a value exists and is blank'.
3 Optional references are null, never undefined, so a record round-trips through JSON without changing shape.
4 The shipping catalogue holds no categories, no specifications, no tags, no assets, no revisions and no users.
5 verify.ts asserts all of this on every run, because 'every field ships empty' is easy to claim and easy to break with one careless default.

M4 Revision history and diffs

Field-level diffs — confirmed. A revision is written automatically on every save, so history is a by-product of authoring rather than a separate chore the author has to remember.

REVISION

`version`	number — monotonic per spec
`createdAt / createdBy`	who and when
`summary`	authored, or generated
`status`	state at the time
`published`	publish event or draft save
`changes`	FieldChange[]
`snapshot`	full record, serialised

FIELDCHANGE — ONE PER ALTERED FIELD

PATH	LABEL	KIND	BEFORE → AFTER
`title`	Title	changed	— → —
`blocks.0.lines.4.text`	Technical · line 5	changed	— → —
`blocks.1.lines.2`	Requirements · line 3	added	— → —
`metadata.effective_date`	Effective date	changed	— → —
`downloadIds.2`	Die line v2.pdf	removed	— → —

WHY BOTH DIFFS AND A SNAPSHOT

Diffs answer 'what changed', which is what the history table shows and what a reviewer needs.

The snapshot answers 'what did it look like', which is what restore needs. Replaying diffs backwards to reconstruct a record is fragile and gets more fragile as the schema evolves.

Storing both costs disk and buys correctness. For a catalogue of hundreds of specifications that is the right trade.

VERSIONING RULES

Specification.version is the published version, starting at 0 and incrementing only on a publish event.

Revision.version is monotonic per specification and increments on every save, published or not.

So a specification at v3 may have twenty revisions behind it — seventeen drafts and three publishes. The downloads bar and the PDF export both carry the published version, never the revision count, so a printed sheet is never mistaken for current.

1 Every save writes a Revision. There is no separate 'record a change' action, because one that exists will not be used.
2 FieldChange carries a dotted path, a human label for the diff view, a kind, and serialised before and after values.
3 Paths reach into nested content — blocks.0.lines.4.text — so a change to one line of technical specification is recorded precisely, not as 'blocks changed'.
4 A full snapshot accompanies the diffs so restore does not depend on replaying history backwards.
5 Published version and revision count are different numbers and are never conflated. Exports carry the published version.

3. Entities

Thirteen entities. Field tables below are the authoritative reference; the TypeScript files are generated from the same decisions, not the other way round.

3.1 Specification

The leaf record. Everything else exists to organise, describe or version one of these. Twelve sections, matching the Phase 1 information architecture and the Phase 2 wireframes.

Field	Type	Empty value	Notes
id	SpecificationId	minted	Branded. Sortable, collision-resistant.
title	string	""	Required before save. The only required text field.
slug	Slug	""	Derived from title. Uniqueness enforced across the catalogue.
code	string	""	Free text, optional, no enforced format — confirmed. Uniqueness is advisory, so an existing external scheme can be entered as-is.
description	string	""	Plain text. Renders under the title and feeds the search index.
categoryId	CategoryId | null	null	Exclusive — exactly one category. Required before save.
subGroupId	SubGroupId | null	null	The nullable middle tier. Must belong to the same category.
tagIds	TagId[]	[]	Many-to-many. The only axis that crosses categories.
status	SpecStatus	'draft'	One of five. Transitions enforced.
supersededById	SpecificationId | null	null	Required when status is 'superseded'. Self-reference rejected.
version	number	0	Published version. Increments only on a publish event.
publishedAt	string | null	null	First publish timestamp.
blocks	SpecBlock[]	[]	Technical specifications, requirements, notes — labels are authored.
metadata	MetadataField[]	[]	Dynamic key/value pairs — confirmed structure.
mediaIds	AssetId[]	[]	The image pane and its thumbnail strip.
downloadIds	AssetId[]	[]	The pinned downloads bar. Die lines, artwork templates.
attachmentIds	AssetId[]	[]	The attachments section. Any file type.
relatedIds	SpecificationId[]	[]	Explicit cross-references. Self-reference rejected.
revisionIds	RevisionId[]	[]	Every save, published or not.
currentRevisionId	RevisionId | null	null	Points at the most recent revision.
searchText	string	""	Derived on save. Never authored.
createdAt / By, updatedAt / By	Timestamp | UserId | null	null	Audit trail.
archivedAt / By	Timestamp | UserId | null	null	Soft deletion. Restorable from the Archive tab.

3.2 Category and SubGroup

The navigable spine. Category is exclusive and ordered; SubGroup is the optional middle tier the reference system already handles with the same navigation primitive.

Entity	Field	Type	Notes
Category	name / slug / description	string	Slug derived from name.
	order	SortOrder	Authored, not alphabetical. Specification catalogues have a conventional sequence.
	mediaId	AssetId | null	Optional category header image.
	subGroupIds	SubGroupId[]	In authored order. Empty means the category exposes specifications directly.
	metadataTemplate	MetadataTemplate	Fields new specifications inherit — with empty values. This is how a domain convention propagates without a schema change.
	blockTemplate	string[]	Block labels new specifications start with.
SubGroup	categoryId	CategoryId	Parent. Orphans are a validation error.
	name / slug / description	string	

Entity	Field	Type	Notes
	order	SortOrder	Drag-reorderable within its category.

3.3 Tag

The cross-cutting axis. Many-to-many, unordered, never navigable as a tree. Keeping tags visually and structurally distinct from categories is what stops the taxonomy collapsing as the catalogue grows.

Field	Type	Notes
name / slug / description	string	
tone	TagTone	Maps to the design-system semantic tokens, so tags can be colour-coded without new palette entries.
usageCount	number	Denormalised for facet counts. Recomputed, never authored.

3.4 SpecBlock and BlockLine

The reference system's core content pattern, modelled explicitly. A block is a titled run of lines; a line is an item or a sub-heading, optionally carrying an inline reference callout.

Entity	Field	Type	Notes
SpecBlock	label	string	Authored. "Technical specifications" is a convention, not schema.
	lines	BlockLine[]	
	twoColumn	boolean	The reference system's two-column toggle, retained.
	order / anchorId	SortOrder / string	anchorId is the scroll-spy target for the anchor rail.
BlockLine	kind	'item' | 'subheading'	The reference system marks sub-headings with a leading **.
	text	string	
	reference	string | null	Inline callout rendered in the accent colour. Written as *[CODE] in the reference system.

3.5 MetadataField

Dynamic key/value pairs — the confirmed structure. Adding a field never requires a code change or a migration, which is the point of the reusable platform.

Field	Type	Notes
key	string	Stable machine key. Survives a label rename, so renaming a field does not orphan its data.
label	string	What the user sees.
type	MetadataValueType	text · number · date · boolean · dimension · url · user. Drives the input control and the sort comparator.
value	string | null	Null means empty. Every field ships null.
unit	string | null	For dimension and number types.
order	SortOrder	Position in the boxed definition grid.
inherited	boolean	True when the field came from the category template rather than being authored on the record.
indexed	boolean	Surfaced in list columns and search facets.

3.6 Asset

One file model with a role, rather than three parallel types. The reference system handles exactly two kinds — one JPEG per specification and a list of die-line PDFs committed through a browser-held repository token.

Field	Type	Notes
role	'media' | 'download' | 'attachment'	Determines where the asset surfaces, not what it is.
kind	AssetKind	image · video · document · cad · archive · other. Derived from MIME type.
filename / storageKey	string	storageKey is opaque. The service layer resolves it to a URL.

Field	Type	Notes
mimeType / sizeBytes	string / number | null	
alt	string	Required for images and video. Used as the accessible name.
caption / order	string / SortOrder	
image	ImageMeta | null	Width, height, blurred placeholder, and a focal point. The reference system hard-codes a 30% focal point on tablet.
version	string | null	Version label for downloads, so a superseded die line is identifiable.

> **STORAGE IS ABSTRACTED**
>
> StorageAdapter is an interface with put, url and remove. Phase 5 ships a local adapter.
>
> The reference system's repository-token strategy becomes one implementation of that interface rather than a property of the product. This is the security exposure flagged in Phase 1 §2.15, resolved structurally.

3.7 Revision and FieldChange

Field-level diffs, confirmed. Written automatically on every save.

Entity	Field	Type	Notes
Revision	version	number	Monotonic per specification. Increments on every save, published or not.
	summary	string	Author-supplied, optional. Falls back to a generated description of the changes.
	status / published	SpecStatus / boolean	State at the time, and whether this was a publish event.
	changes	FieldChange[]	What changed.
	snapshot	string | null	The full record, serialised. What restore uses.
FieldChange	path	FieldPath	Dotted path — blocks.0.lines.4.text. Precise to the line.
	label	string	Human label for the diff view — "Technical · line 5".
	kind	'added' | 'removed' | 'changed'	
	before / after	string | null	Null on added and removed respectively.

3.8 User and Role

Four roles, declared now so Approver exists before approval workflows are built rather than being retrofitted.

Role	Permissions
viewer	spec.read
editor	spec.read · spec.create · spec.update · spec.archive
approver	the above, plus spec.approve · spec.publish
administrator	the above, plus spec.delete · taxonomy.manage · user.manage · settings.manage

Permissions are a flat list checked by can(role, permission). Note that only administrators hold spec.delete — permanent deletion exists solely in the Archive tab and requires a typed confirmation, as specified on Phase 2 plate D13.

3.9 Collections

Bookmarks, favourites and recently viewed. Derived, device-scoped, never authored and never part of the category tree. Kept in a separate adapter so they can move to an account later without touching the content model.

3.10 Search and filter

A pre-built index over every searchable field. The reference system filters the sidebar tree by title substring and has no results surface at all.

Type	Purpose
IndexEntry	Title, code, description, tags, status and concatenated body text, plus a normalised form for prefix matching. Built once, queried many times.
FilterState	Every field maps to a URL parameter, so a filtered view is shareable and survives reload — the Phase 1 §5.2 requirement.
Facets	Live counts per category, sub-group, tag and status, driving the filter rail's count column.
SearchResult	Score plus character ranges to highlight, per matched field.

4. Invariants

Rules checked before anything is persisted. Each is a function in invariants.ts and each has a test in verify.ts.

Rule	Code	Why
A specification must have a title	`required`	The only required text field. Everything else may legitimately stay empty.
A specification must belong to exactly one category	`required`	Category is the exclusive primary axis — confirmed.
A sub-group must belong to its specification's category	`orphaned`	Prevents a record appearing under a branch it does not belong to.
Referenced categories, sub-groups, tags and assets must exist	`not_found`	Catches dangling references before they become rendering failures.
A superseded specification must name its replacement	`missing_replacement`	This is the state that matters most for print specifications, where retired versions circulate physically.
A specification cannot supersede or relate to itself	`self_reference`	Would produce an infinite navigation loop.
Metadata keys must be unique within a specification	`duplicate`	Two fields with the same key make the record ambiguous.
Slugs must be unique across the catalogue	`duplicate`	Slugs appear in URLs.
Status transitions must be permitted	`illegal_transition`	Seven permitted, thirteen blocked. Nothing returns to draft once published.
Revisions must belong to an existing specification	`orphaned`	History without a subject is unrecoverable.
Deleting a category with specifications requires reassignment	—	categoryDeletionBlockers() returns the affected records. Never cascades.

5. The empty catalogue

What the application loads on first run, and how that is guaranteed.

Check	Result
Schema version declared	**PASS v1**
A fresh catalogue is empty	**PASS**
The shipping seed is empty	**PASS**
Seed holds no assets, revisions or users	**PASS**
Seed search index is unbuilt	**PASS**
New Specification carries no content	**PASS**
New Category, SubGroup, Tag, Asset, User carry no content	**PASS**
Category inheritance adds no values	**PASS**
Empty catalogue validates clean	**PASS**
A specification without a category is rejected	**PASS**
Self-relation is rejected	**PASS**
Superseded without a replacement is rejected	**PASS**
Transition map is restrictive	**PASS 7 permitted, 13 blocked**
Nothing returns to draft from published	**PASS**
Superseded is terminal	**PASS**
Terminal flags match the transition map	**PASS**
An illegal transition returns an error	**PASS**
A legal transition succeeds	**PASS**
TypeScript strict compilation	**PASS 0 errors**

WHY THIS IS A SCRIPT AND NOT A SENTENCE

"Every field ships empty" is trivially easy to claim and trivially easy to break — one default string, one seeded category, one convenience value, and the promise is gone without anyone noticing.

verify.ts walks every factory recursively and fails the build if any field carries authored content. It caught one genuine problem while being written: an over-strict check that flagged an empty nested object as populated.

6. Extensibility

Your Phase 9 requirement was that future additions need no structural change. This is how each named item is already accommodated.

Phase 9 requirement	How the model already carries it
Hundreds of specifications	Normalised storage keyed by id, a pre-built search index, and a summary type so a list row never loads blocks or assets.
Multiple categories	Category is a first-class entity with authored order and its own templates. Adding one is data, not code.
Nested specifications	The nullable SubGroup tier handles one level. Deeper nesting would need a parentId on Category — a single field, no restructuring.
Images, videos, PDFs, CAD files	One Asset model with a kind derived from MIME type. AssetKind already names all four.
Downloads	AssetRole distinguishes downloads from media and attachments, so the pinned bar is a filter, not a separate collection.
Version control	Revision with field-level diffs and a full snapshot. Published version and revision count are distinct numbers.
User permissions	Four roles, a flat permission list, and can(). Adding a permission is one entry in the map.
Approval workflows	The in_review state and the approver role both exist now. A workflow adds transitions, not entities.
Search indexing	SearchIndex, IndexEntry, Facets and FilterState are defined. Swapping in an external index means implementing one interface.
API integration	Nothing in the model references a transport. The service layer in Phase 5 is the only thing that knows where data comes from.
CMS integration	Same. A CMS becomes an adapter that returns a Catalogue.

6.1 What would require a migration

Stated plainly, because a model that claims to accommodate everything is not being honest.

- Making category non-exclusive. Changing categoryId to categoryIds would touch the navigation, the filter rail, the breadcrumb and the index. This is why the question was worth asking before building.
- Nesting categories more than one level deep. The tree components assume a fixed depth of three.
- Making metadata typed rather than dynamic. Going the other way — dynamic to fixed — is the migration; the confirmed direction avoids it.

SCHEMA_VERSION exists for exactly these cases. Phase 5 reads it on load and can run a migration when it does not match.

7. Delivered files

data-model/ in your project folder. No runtime dependencies.

File	Contains
src/ids.ts	Branded identifier types, opaque constructors, and id minting.
src/common.ts	Timestamps, audit and archive mixins, Result and ValidationError, slugify.
src/status.ts	The five states, their metadata, and the transition map.
src/content.ts	SpecBlock, BlockLine, MetadataField, MetadataTemplate.
src/assets.ts	Asset, ImageMeta, StorageAdapter, MIME-to-kind mapping.
src/taxonomy.ts	Category, SubGroup, Tag.
src/specification.ts	Specification and SpecificationSummary.
src/revision.ts	Revision, FieldChange, generated change descriptions.
src/access.ts	User, Role, Permission, ROLE_PERMISSIONS, can().
src/collections.ts	Bookmarks, favourites, recently viewed, and their adapter.
src/search.ts	SearchIndex, IndexEntry, FilterState, Facets, SearchResult.
src/catalogue.ts	The Catalogue aggregate, SCHEMA_VERSION, emptyCatalogue, isEmpty.
src/factories.ts	Empty-value constructors, including category-template inheritance.
src/invariants.ts	validateSpecification, validateTransition, validateCatalogue, categoryDeletionBlockers.
src/index.ts	Barrel export.
data/catalogue.empty.ts	The shipping catalogue. Verifiably empty.
verify.ts	23 assertions covering emptiness, invariants and the lifecycle.
tsconfig.json · package.json	Strict configuration. npm run check runs both the type-checker and the assertions.

8. Phase 5 scope & approval

On approval, Phase 5 produces the application architecture — deliverable 10. The project structure, the service layer, routing, state and the build configuration.

8.1 Scope

- **Project structure —** React, TypeScript and Vite with the directory layout your brief specifies — components, layouts, pages, hooks, utils, types, services, data, assets, styles — populated with the fifty-nine components from Phase 3 and this data model.
- **Service layer —** The adapter boundary. A local file adapter backed by data/catalogue.empty.ts, written so an API or CMS can replace it without touching a component.
- **Routing —** Hash routing as specified in Phase 1 §6, isolated behind one module so a switch to history routing is a single-file change.
- **State —** What lives in the URL (filters, sort, view), what lives in memory (catalogue, index), and what persists per device (collections, theme, view preference).
- **Build —** Vite configuration, path aliases, the Tailwind bridge to the Phase 3 tokens, and a static build that deploys to GitHub Pages exactly as the current site does.

8.2 Decisions needed

- Package manager and Node version, if you have a preference. Otherwise npm and Node 20 LTS.
- Whether the admin layer should be behind a build flag, so a public deployment can ship without it.
- Whether to include a test setup now — Vitest and Testing Library — or defer it to Phase 6.

8.3 Approval

This is the last phase where the shape of the data can change cheaply. Once components are written against these types, changing categoryId to categoryIds or making metadata a fixed schema means touching the navigation, the filters, the breadcrumb and the index together.

End of Phase 4. Deliverable 9 of 12 complete. Awaiting approval to proceed to Phase 5.